1. Qual é o problema?

Duas palavras e queremos a maior sequência de letras que aparece nas duas **na mesma ordem**, podendo pular letras.

Com X = ABCB e Y = BCAB, a resposta é BCB. Em Y aparece B, C, depois um A que pulamos, e depois B

Pular é permitido, trocar a ordem não é.

Substring = recortar com tesoura

Por exemplo: Na palavra BRASIL

"BRA" é substring, porque é o pedaço do começo.

"ASI" é substring, porque é o pedaço do meio.

As **letras têm que estar coladas**, uma do lado da outra, exatamente como estão na palavra.

Subsequência = riscar letras com caneta

Pode **riscar as letras que quiser**, e as que sobram formam a subsequência. A única regra é que **não pode mudar a ordem delas**.

Na palavra BRASIL:

BRASIL

B-A-L -> é subsequência

"LAB" ✗. As letras L, A e B existem em BRASIL, mas na palavra a ordem é B, depois A, depois L. Para formar "LAB" você teria que mudar as letras de lugar, e isso não pode.

Portanto:

Substring: letras coladas.
Subsequência: letras podem estar separadas, mas na mesma ordem.

Um exemplo de utilidade:

git diff - Quando você compara duas versões de um código, o Git procura a maior sequência de linhas que se manteve igual. As linhas que sobram são as que foram adicionadas ou removidas. É o LCS aplicado a linhas em vez de letras.

2. O que o algoritmo resolve?

Entrada: duas strings, X com m letras e Y com n Letras.

Saída: o tamanho da LCS e a própria subsequência.

Objetivo: encontrar a maior sequência comum, **na mesma ordem**.

3. Qual é a estratégia?

a) Por que não força bruta? A força bruta testaria todas as subsequências de X. Para cada letra existem duas opções, entra ou não entra, então são 2 × 2 × 2 × ... = **2^m** possibilidades. Cada letra a mais dobra o trabalho. Com 30 letras já passa de 1 bilhão.

b) A ideia-chave da PD: olhar para a última letra. Pense em resolver o problema para dois prefixos, por exemplo "ABC" e "BCA". Olhe só para a última letra de cada um:

- **Se as últimas letras são iguais**, essa letra com certeza pode entrar na resposta. Então a resposta é a LCS do que sobra, sem essas duas letras, **mais 1**.
- **Se são diferentes**, pelo menos uma das duas não entra na resposta. Como não sabemos qual, testamos as duas opções e ficamos com a melhor:
    - tirar a última letra de X, ou
    - tirar a última letra de Y.

Essa é a justificativa da regra, e é bom saber explicar com essas palavras. A regra não foi inventada: ela vem desse raciocínio sobre a última letra.

c) As duas condições da programação dinâmica:

1. **Subestrutura ótima:** a resposta do problema grande é montada com as respostas de problemas menores (prefixos menores). Foi exatamente o que fizemos em (b).
2. **Subproblemas sobrepostos:** os mesmos subproblemas aparecem várias vezes. Por exemplo, LCS ("ABC", "BCA") com letras diferentes pede LCS ("AB", "BCA") e LCS ("ABC", "BC"). **As duas** acabam pedindo LCS ("AB", "BC"). Uma recursão simples calcularia isso duas vezes, e o desperdício cresce de forma exponencial. A PD calcula **uma vez** e guarda na tabela.

Por que não é guloso? O guloso pega a primeira letra igual que encontra e nunca volta atrás. No nosso exemplo, se ele casar o A de X com o A de Y, depois do A em Y só sobra um B, e ele termina com "AB" (tamanho 2). A resposta certa tem tamanho 3. A escolha que parecia boa no momento estragou o resultado.

Por que não divisão e conquista? A divisão e conquista quebra o problema em partes **independentes**, como o merge sort, que ordena cada metade separadamente. Aqui os

subproblemas **se repetem**, então resolver cada um separadamente seria refazer trabalho. A diferença está na palavra “sobrepostos”.

4. Como o algoritmo funciona?

A tabela. Cada célula dp[i][j] responde uma pergunta: *“qual o tamanho da LCS entre as i primeiras letras de X e as j primeiras letras de Y?”*. A tabela inteira é um mapa com a resposta de todos os subproblemas.

A borda com zeros. A linha 0 e a coluna 0 representam a palavra vazia, e comparar qualquer coisa com uma palavra vazia dá 0. É por isso que a tabela tem tamanho (m+1) × (n+1).

A regra, traduzida para a tabela. Aqui o raciocínio do ponto 3 vira posição na tabela:

- **Diagonal** (dp[i-1] [j-1]) = sem a última letra de X **e** sem a de Y.
- **Cima** (dp[i-1] [j]) = sem a última letra de X.
- **Esquerda** (dp[i][j-1]) = sem a última letra de Y.

Então, se as letras são iguais, usamos diagonal + 1. Se são diferentes, usamos o maior entre cima e esquerda.

Por que preencher linha por linha, da esquerda para a direita? Porque cada célula depende da diagonal, de cima e da esquerda. Nessa ordem, quando chegamos numa célula, essas três já foram calculadas.

O backtracking (montar a palavra). A tabela guarda só números. Para descobrir as letras, voltamos do canto inferior direito refazendo o caminho de onde cada número veio:

- Letras iguais: o número veio da diagonal, então essa letra faz parte da resposta. Guardamos e andamos na diagonal.
- Letras diferentes: o número veio do vizinho maior, então andamos para ele.

As letras saem de trás para frente, e por isso cada letra nova é colocada **no começo** da string.

5. Hands-on

O professor quer ver o código **nascer** na frente da turma. A dinâmica é perguntar antes de digitar.

1. **Estrutura mínima:** a função e a tabela de zeros. Rode e mostre a matriz 5×5 de zeros, que é a tabela do quadro.
2. **A regra:** os dois for e o if/else. Antes de digitar o if, pergunte: “letras iguais, o que fazemos?”. Rode e veja aparecer o 3.

3. **Mostrar a tabela:** imprima e compare com a tabela do quadro, número por número.
4. **Backtracking:** o while. Rode e veja aparecer “BCB”.

Dois detalhes que costumam confundir a turma e que você deve saber explicar:

- **x [i - 1]:** a tabela começa no índice 1, porque o 0 é a palavra vazia, mas a string começa no índice 0. A linha i da tabela corresponde à letra x[i-1].
- **range (1, m + 1):** começa em 1 porque a linha 0 já está pronta, toda com zero. Vai até m + 1 porque o range não inclui o último número.

6. Teste

Cada teste tem um motivo, e vale dizer qual é:

Teste	**O que ele verifica**
ABCB / BCAB → BCB (3)	O exemplo da apresentação: confere com o quadro
AGGTAB / GXTXAYB → GTAB (4)	Palavras de tamanhos diferentes e um caso maior
ABC / DEF → “” (0)	Caso limite: nada em comum, a tabela fica toda com zero

Testar só o caso que funciona não prova muito. O terceiro teste mostra que o código não quebra quando não há resposta.

7. Complexidade

Para justificar, conte o que o código faz:

- **Tempo O (m × n):** o for de fora roda m vezes e o de dentro roda n vezes para cada volta do de fora. Lá dentro há só uma comparação e uma conta, o que é tempo constante. Total: m × n passos. O backtracking anda no máximo m + n passos, porque a cada passo i ou j diminui, e isso é pequeno perto de m × n.
- **Espaço O (m × n):** é o tamanho da tabela.

A comparação que impressiona é com m = n = 30: a PD faz cerca de **900** operações, enquanto a força bruta faz 2^30 × 30, **mais de 32 bilhões**. A frase para fechar: *a programação dinâmica troca tempo exponencial por tempo polinomial, gastando memória com a tabela.*